

9.2 Implementing a Priority Queue

In this section, we discuss several technical issues involving the implementation of the priority queue ADT in Java, and we define an abstract base class that provides functionality that is shared by all priority queue implementations in this chapter. We then provide two concrete priority queue implementations using a positional list *L* (see Section 7.3) for storage. They differ in whether or not entries are maintained in sorted order according to their keys.

9.2.1 The Entry Composite

One challenge in implementing a priority queue is that we must keep track of both an element and its key, even as entries are relocated within a data structure. This is reminiscent of a case study from Section 7.7 in which we maintain a list of elements with access frequencies. In that setting, we introduced the *composition design pattern*, defining an *Item* class that paired each element with its associated count in our primary data structure. For priority queues, we use composition to pair a key *k* and a value *v* as a single object. To formalize this, we define the public interface, *Entry*, shown in Code Fragment 9.1.

```

1  /** Interface for a key-value pair. */
2  public interface Entry<K,V> {
3      K getKey();                // returns the key stored in this entry
4      V getValue();              // returns the value stored in this entry
5  }
```

Code Fragment 9.1: Java interface for an entry storing a key-value pair.

We then use the *Entry* type in the formal interface for the priority queue, shown in Code Fragment 9.2. This allows us to return both a key and value as a single object from methods such as *min* and *removeMin*. We also define the *insert* method to return an entry; in a more advanced *adaptable priority queue* (see Section 9.5), that entry can be subsequently updated or removed.

```

1  /** Interface for the priority queue ADT. */
2  public interface PriorityQueue<K,V> {
3      int size();
4      boolean isEmpty();
5      Entry<K,V> insert(K key, V value) throws IllegalArgumentException;
6      Entry<K,V> min();
7      Entry<K,V> removeMin();
8  }
```

Code Fragment 9.2: Java interface for the priority queue ADT.

9.2.2 Comparing Keys with Total Orders

In defining the priority queue ADT, we can allow any type of object to serve as a key, but we must be able to compare keys to each other in a meaningful way. More so, the results of the comparisons must not be contradictory. For a comparison rule, which we denote by $\leq$, to be self-consistent, it must define a **total order** relation, which is to say that it satisfies the following properties for any keys k_1 , k_2 , and k_3 :

- **Comparability property:** $k_1 \leq k_2$ or $k_2 \leq k_1$.
- **Antisymmetric property:** if $k_1 \leq k_2$ and $k_2 \leq k_1$, then $k_1 = k_2$.
- **Transitive property:** if $k_1 \leq k_2$ and $k_2 \leq k_3$, then $k_1 \leq k_3$.

The comparability property states that comparison rule is defined for every pair of keys. Note that this property implies the following one:

- **Reflexive property:** $k \leq k$.

A comparison rule, $\leq$, that defines a total order relation will never lead to a contradiction. Such a rule defines a linear ordering among a set of keys; hence, if a (finite) set of elements has a total order defined for it, then the notion of a **minimal** key, $k_{\min}$, is well defined, as a key in which $k_{\min} \leq k$, for any other key k in our set.

The Comparable Interface

Java provides two means for defining comparisons between object types. The first of these is that a class may define what is known as the **natural ordering** of its instances by formally implementing the `java.lang.Comparable` interface, which includes a single method, `compareTo`. The syntax `a.compareTo(b)` must return an integer i with the following meaning:

- $i < 0$ designates that $a < b$.
- $i = 0$ designates that $a = b$.
- $i > 0$ designates that $a > b$.

For example, the `compareTo` method of the `String` class defines the natural ordering of strings to be **lexicographic**, which is a case-sensitive extension of the alphabetic ordering to Unicode.

The Comparator Interface

In some applications, we may want to compare objects according to some notion other than their natural ordering. For example, we might be interested in which of two strings is the shortest, or in defining our own complex rules for judging which of two stocks is more promising. To support generality, Java defines the `java.util.Comparator` interface. A **comparator** is an object that is external to the class of the keys it compares. It provides a method with the signature `compare(a, b)` that returns an integer with similar meaning to the `compareTo` method described above.

As a concrete example, Code Fragment 9.3 defines a comparator that evaluates strings based on their length (rather than their natural lexicographic order).

```

1 public class StringLengthComparator implements Comparator<String> {
2     /** Compares two strings according to their lengths. */
3     public int compare(String a, String b) {
4         if (a.length() < b.length()) return -1;
5         else if (a.length() == b.length()) return 0;
6         else return 1;
7     }
8 }

```

Code Fragment 9.3: A comparator that evaluates strings based on their lengths.

Comparators and the Priority Queue ADT

For a general and reusable form of a priority queue, we allow a user to choose any key type and to send an appropriate comparator instance as a parameter to the priority queue constructor. The priority queue will use that comparator anytime it needs to compare two keys to each other.

For convenience, we also allow a default priority queue to instead rely on the natural ordering for the given keys (assuming those keys come from a comparable class). In that case, we build our own instance of a `DefaultComparator` class, shown in Code Fragment 9.4.

```

1 public class DefaultComparator<E> implements Comparator<E> {
2     public int compare(E a, E b) throws ClassCastException {
3         return ((Comparable<E>) a).compareTo(b);
4     }
5 }

```

Code Fragment 9.4: A `DefaultComparator` class that implements a comparator based upon the natural ordering of its element type.

9.2.3 The `AbstractPriorityQueue` Base Class

To manage technical issues common to all our priority queue implementations, we define an abstract base class named `AbstractPriorityQueue` in Code Fragment 9.5. (See Section 2.3.3 for a discussion of abstract base classes.) This includes a nested `PQEntry` class that implements the public `Entry` interface.

Our abstract class also declares and initializes an instance variable, `comp`, that stores the comparator being used for the priority queue. We then provide a protected method, `compare`, that invokes the comparator on the keys of two given entries.

```

1  /** An abstract base class to assist implementations of the PriorityQueue interface.*/
2  public abstract class AbstractPriorityQueue<K,V>
3                                     implements PriorityQueue<K,V> {
4      //----- nested PQEntry class -----
5      protected static class PQEntry<K,V> implements Entry<K,V> {
6          private K k; // key
7          private V v; // value
8          public PQEntry(K key, V value) {
9              k = key;
10             v = value;
11         }
12         // methods of the Entry interface
13         public K getKey() { return k; }
14         public V getValue() { return v; }
15         // utilities not exposed as part of the Entry interface
16         protected void setKey(K key) { k = key; }
17         protected void setValue(V value) { v = value; }
18     } //----- end of nested PQEntry class -----
19
20     // instance variable for an AbstractPriorityQueue
21     /** The comparator defining the ordering of keys in the priority queue. */
22     private Comparator<K> comp;
23     /** Creates an empty priority queue using the given comparator to order keys. */
24     protected AbstractPriorityQueue(Comparator<K> c) { comp = c; }
25     /** Creates an empty priority queue based on the natural ordering of its keys. */
26     protected AbstractPriorityQueue() { this(new DefaultComparator<K>()); }
27     /** Method for comparing two entries according to key */
28     protected int compare(Entry<K,V> a, Entry<K,V> b) {
29         return comp.compare(a.getKey(), b.getKey());
30     }
31     /** Determines whether a key is valid. */
32     protected boolean checkKey(K key) throws IllegalArgumentException {
33         try {
34             return (comp.compare(key,key) == 0); // see if key can be compared to itself
35         } catch (ClassCastException e) {
36             throw new IllegalArgumentException("Incompatible key");
37         }
38     }
39     /** Tests whether the priority queue is empty. */
40     public boolean isEmpty() { return size() == 0; }
41 }

```

Code Fragment 9.5: The `AbstractPriorityQueue` class. This provides a nested `PQEntry` class that composes a key and a value into a single object, and support for managing a comparator. For convenience, we also provide an implementation of `isEmpty` based on a presumed `size` method.

9.2.4 Implementing a Priority Queue with an Unsorted List

In our first concrete implementation of a priority queue, we store entries within an *unsorted* linked list. Code Fragment 9.6 presents our `UnsortedPriorityQueue` class as a subclass of the `AbstractPriorityQueue` class (from Code Fragment 9.5). For internal storage, key-value pairs are represented as composites, using instances of the inherited `PQEntry` class. These entries are stored within a `PositionalList` that is an instance variable. We assume that the positional list is implemented with a doubly linked list, as in Section 7.3, so that all operations of that ADT execute in $O(1)$ time.

We begin with an empty list when a new priority queue is constructed. At all times, the size of the list equals the number of key-value pairs currently stored in the priority queue. For this reason, our priority queue size method simply returns the length of the internal list. By the design of our `AbstractPriorityQueue` class, we inherit a concrete implementation of the `isEmpty` method that relies on a call to our size method.

Each time a key-value pair is added to the priority queue, via the insert method, we create a new `PQEntry` composite for the given key and value, and add that entry to the end of the list. Such an implementation takes $O(1)$ time.

The remaining challenge is that when `min` or `removeMin` is called, we must locate the entry with minimal key. Because the entries are not sorted, we must inspect all entries to find one with a minimal key. For convenience, we define a private `findMin` utility that returns the *position* of an entry with minimal key. Knowledge of the position allows the `removeMin` method to invoke the `remove` method on the positional list. The `min` method simply uses the position to retrieve the entry when preparing a key-value tuple to return. Due to the loop for finding the minimal key, both `min` and `removeMin` methods run in $O(n)$ time, where n is the number of entries in the priority queue.

A summary of the running times for the `UnsortedPriorityQueue` class is given in Table 9.1.

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
insert	$O(1)$
min	$O(n)$
removeMin	$O(n)$

Table 9.1: Worst-case running times of the methods of a priority queue of size n , realized by means of an unsorted, doubly linked list. The space requirement is $O(n)$.

```

1  /** An implementation of a priority queue with an unsorted list. */
2  public class UnsortedPriorityQueue<K,V> extends AbstractPriorityQueue<K,V> {
3      /** primary collection of priority queue entries */
4      private PositionalList<Entry<K,V>> list = new LinkedPositionalList<>();
5
6      /** Creates an empty priority queue based on the natural ordering of its keys. */
7      public UnsortedPriorityQueue() { super(); }
8      /** Creates an empty priority queue using the given comparator to order keys. */
9      public UnsortedPriorityQueue(Comparator<K> comp) { super(comp); }
10
11     /** Returns the Position of an entry having minimal key. */
12     private Position<Entry<K,V>> findMin() { // only called when nonempty
13         Position<Entry<K,V>> small = list.first();
14         for (Position<Entry<K,V>> walk : list.positions())
15             if (compare(walk.getElement(), small.getElement()) < 0)
16                 small = walk; // found an even smaller key
17         return small;
18     }
19
20     /** Inserts a key-value pair and returns the entry created. */
21     public Entry<K,V> insert(K key, V value) throws IllegalArgumentException {
22         checkKey(key); // auxiliary key-checking method (could throw exception)
23         Entry<K,V> newest = new PQEntry<>(key, value);
24         list.addLast(newest);
25         return newest;
26     }
27
28     /** Returns (but does not remove) an entry with minimal key. */
29     public Entry<K,V> min() {
30         if (list.isEmpty()) return null;
31         return findMin().getElement();
32     }
33
34     /** Removes and returns an entry with minimal key. */
35     public Entry<K,V> removeMin() {
36         if (list.isEmpty()) return null;
37         return list.remove(findMin());
38     }
39
40     /** Returns the number of items in the priority queue. */
41     public int size() { return list.size(); }
42 }

```

Code Fragment 9.6: An implementation of a priority queue using an unsorted list. The parent class `AbstractPriorityQueue` is given in Code Fragment 9.5, and the `LinkedPositionalList` class is from Section 7.3.

9.2.5 Implementing a Priority Queue with a Sorted List

Our next implementation of a priority queue also uses a positional list, yet maintains entries sorted by nondecreasing keys. This ensures that the first element of the list is an entry with the smallest key.

Our SortedPriorityQueue class is given in Code Fragment 9.7. The implementation of min and removeMin are rather straightforward given knowledge that the first element of a list has a minimal key. We rely on the first method of the positional list to find the position of the first entry, and the remove method to remove the entry from the list. Assuming that the list is implemented with a doubly linked list, operations min and removeMin take $O(1)$ time.

This benefit comes at a cost, however, for method insert now requires that we scan the list to find the appropriate position to insert the new entry. Our implementation starts at the end of the list, walking backward until the new key is smaller than that of an existing entry; in the worst case, it progresses until reaching the front of the list. Therefore, the insert method takes $O(n)$ worst-case time, where n is the number of entries in the priority queue at the time the method is executed. In summary, when using a sorted list to implement a priority queue, insertion runs in linear time, whereas finding and removing the minimum can be done in constant time.

Comparing the Two List-Based Implementations

Table 9.2 compares the running times of the methods of a priority queue realized by means of a sorted and unsorted list, respectively. We see an interesting trade-off when we use a list to implement the priority queue ADT. An unsorted list supports fast insertions but slow queries and deletions, whereas a sorted list allows fast queries and deletions, but slow insertions.

Method	Unsorted List	Sorted List
size	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
insert	$O(1)$	$O(n)$
min	$O(n)$	$O(1)$
removeMin	$O(n)$	$O(1)$

Table 9.2: Worst-case running times of the methods of a priority queue of size n , realized by means of an unsorted or sorted list, respectively. We assume that the list is implemented by a doubly linked list. The space requirement is $O(n)$.

```

1  /** An implementation of a priority queue with a sorted list. */
2  public class SortedPriorityQueue<K,V> extends AbstractPriorityQueue<K,V> {
3      /** primary collection of priority queue entries */
4      private PositionalList<Entry<K,V>> list = new LinkedPositionalList<>();
5
6      /** Creates an empty priority queue based on the natural ordering of its keys. */
7      public SortedPriorityQueue() { super(); }
8      /** Creates an empty priority queue using the given comparator to order keys. */
9      public SortedPriorityQueue(Comparator<K> comp) { super(comp); }
10
11     /** Inserts a key-value pair and returns the entry created. */
12     public Entry<K,V> insert(K key, V value) throws IllegalArgumentException {
13         checkKey(key);    // auxiliary key-checking method (could throw exception)
14         Entry<K,V> newest = new PQEntry<>(key, value);
15         Position<Entry<K,V>> walk = list.last();
16         // walk backward, looking for smaller key
17         while (walk != null && compare(newest, walk.getElement()) < 0)
18             walk = list.before(walk);
19         if (walk == null)
20             list.addFirst(newest);                // new key is smallest
21         else
22             list.addAfter(walk, newest);           // newest goes after walk
23         return newest;
24     }
25
26     /** Returns (but does not remove) an entry with minimal key. */
27     public Entry<K,V> min() {
28         if (list.isEmpty()) return null;
29         return list.first().getElement();
30     }
31
32     /** Removes and returns an entry with minimal key. */
33     public Entry<K,V> removeMin() {
34         if (list.isEmpty()) return null;
35         return list.remove(list.first());
36     }
37
38     /** Returns the number of items in the priority queue. */
39     public int size() { return list.size(); }
40 }

```

Code Fragment 9.7: An implementation of a priority queue using a sorted list. The parent class `AbstractPriorityQueue` is given in Code Fragment 9.5, and the `LinkedPositionalList` class is from Section 7.3.